

Résolution du puzzle « Easy as ABC » et de ses variantes

Le devoir est à travailler et à rendre en binôme. Le rendu comportera une archive (zip ou tar uniquement) avec le code, des tests, un readme, un makefile et un petit document (au format pdf) expliquant vos choix et vos résultats. Vous déposerez vos archives sur exam.ensiie.fr, le nom du dépôt étant SatFisa, le 18 mai 2026 minuit au plus tard. Les noms des deux membres du binôme devront apparaître clairement.

Les programmes demandés peuvent être écrits en C, Java ou Python. En C vous utiliserez glucose et en Java Sat4J. En python vous utiliserez python sat qui permet d'appeler un grand nombre de SAT solveurs (glucose, radical, etc.).

L'objectif de ce mini-projet est de résoudre le puzzle « Easy as ABC » en modélisant le puzzle comme un problème SAT, soit un ensemble de clauses dont on cherche si il est satisfaisable.

Le principe général consiste à remplir une grille vide avec les lettres de l'alphabet (souvent de **A jusqu'à une certaine lettre**, par exemple A à D ou A à I, selon la taille de la grille). Il s'agit de placer chaque lettre **une seule fois par ligne et par colonne**, de façon à respecter les indications placés sur les bords de la grille. Le site <https://gridpuzzle.com/> permet de jouer à ce jeu et à ses différentes variantes.

Pour chacune des variantes ci-dessous, Vous écrirez un programme (ou une fonction ou une méthode) qui prend en entrée la grille de départ et donne une solution.

Plus exactement, vous écrirez :

- un programme qui récupère une entrée définissant une instance du puzzle, et qui crée un fichier DIMACS représentant cette instance ; il est vivement conseillé (pour faciliter les tests) de récupérer les entrées dans un fichier dont vous aurez défini le format ;
- un programme qui, à partir de la trace produite par le SAT solveur choisi, affichera la solution du problème, à savoir la grille remplie ;
- un programme qui enchaîne automatiquement ces étapes ainsi que l'appel au SAT solveur ;
- et un programme qui donne éventuellement une deuxième solution si elle existe (pour obtenir une deuxième solution, on nie la première (il suffit de nier chacun des littéraux de la solution initiale, ce qui nous donne une clause qu'on ajoute à l'ensemble initial de clauses, voir diapositive ajoutée en fin du cours sur le site web4.ensiie.fr/~dubois/LOMA).

Vous pouvez soit appeler directement le SAT solveur depuis votre programme ou réaliser un programme shell qui enchaîne les outils.

Vous testerez vos programmes sur plusieurs instances d'une taille donnée et ferez varier la taille. Dans un premier temps vous testerez des instances données et dans un second temps vous générerez les instances automatiquement.

Indications :

- vous utiliserez un ensemble de variables propositionnelles X_{ijl} pour i compris entre 1 et n , j compris entre 1 et n et l une lettre possible. X_{ijl} vaut vrai quand dans la case à la croisée de la ligne i et de la colonne j se trouve la lettre l .
- Il est conseillé d'écrire une fonction qui transforme les contraintes atleast et atmost en clauses.

1. Variante basique : Easy-as-abc Voir <https://gridpuzzle.com/easy-as-abc>

On part d'une grille carrée $n \times n$ dont les cases doivent être remplies par une série de n lettres (par exemple, de A à F, si $n=6$), chaque lettre devant apparaître exactement une fois dans chaque ligne et chaque colonne. Les lettres situées à l'extérieur de la grille indiquent quelle lettre apparaîtra en premier dans cette direction. De plus certaines cases peuvent être fixées initialement comme ci-dessous à gauche. A droite se trouve la solution.

	C		F	D	A	
.						A
E						
A		F				E
D				A		B
F			B			D
.						F
	C	A	B		F	

		C		F		D		A	
.		B	C	E	F	D	A		A
E		E	B	D	A	F	C		
A		A	F	C	D	B	E		E
D		D	E	F	C	A	B		B
F		F	A	B	E	C	D		D
.		C	D	A	B	E	F		F
		C		A	B		F		

Easy(1) : voir <https://gridpuzzle.com/easy-as-abc1>

On part d'une grille carrée $n \times n$ **vide** dont les cases doivent être remplies par une série de $n-1$ lettres (par exemple, de A à E, si $n=6$), chaque lettre devant apparaître exactement une fois dans chaque ligne et chaque colonne. Chaque ligne et chaque colonne contiennent une case vide (matérialisée par une croix ci-dessous). Les lettres situées à l'extérieur de la grille indiquent quelle lettre apparaîtra en premier dans cette direction, la case vide ne comptant pas.

		B	A	A	E	
D						E
C						A
E						D
						C
	B	A	C	C		

			B	A	A	E	
.	C	D	B	A	×	E	F
D	D	B	C	E	A	×	A
C	×	C	E	D	B	A	
E	E	×	A	B	C	D	D
.	A	E	×	C	D	B	
.	B	A	D	×	E	C	C
	B	A		C		C	

2. Easy(2) : voir <https://gridpuzzle.com/easy-as-abc1>

On part d'une grille carrée $n \times n$ **vide** dont les cases doivent être remplies par une série de $n-2$ lettres (par exemple, de A à D, si $n=6$), chaque lettre devant apparaître exactement une fois dans chaque ligne et chaque colonne. Chaque ligne et chaque colonne contiennent deux cases vides (matérialisées par une croix ci-dessous). Les lettres situées à l'extérieur de la grille indiquent quelle lettre apparaîtra en premier dans cette direction, les cases vides ne comptant pas.

	C	A	A	D	B	
						B
D						C
A						
C						
						D
						A
	B			C	D	

	C	A	A		D	B
C	C	×	A	×	D	B
D	D	×	×	B	A	C
A	×	A	D	C	B	×
C	×	C	B	D	×	A
	A	B	×	×	C	D
	B	D	C	A	×	×